

Informe Preliminar del Proyecto “Sweet Crush”

Tanya Delgado

Facultad de Ingeniería, Universidad de Antioquia

Informática II

Aníbal Guerra

11 de septiembre de 2026

Introducción

La propuesta de desafío contempla el desarrollo de un juego en C++, en donde se manipulen las fichas pertenecientes a un tablero, de modo que puedan eliminarse una a una, hacerlo a nivel de columna o fila y agregar estas, generando nuevas fichas en los espacios con un método de cascada; con los movimientos se busca detectar combinaciones de tres o más fichas de manera vertical u horizontal, lo que causaría que se eliminaran y volviera a rellenar los espacios liberados del tablero de manera aleatoria siendo cada ficha es equiprobable.

Además, el almacenamiento de cada ficha se hará a nivel de bits, siendo el tablero representado por una secuencia de estos, por lo que el uso de la memoria dinámica y la manipulación de bits será clave en el desarrollo.

Análisis para el Desarrollo

Teniendo en cuenta los requerimientos iniciales, se determinó la siguiente codificación para la representación de fichas y su almacenamiento en bits:

<i>Símbolo de la Ficha</i>	<i>Código Binario</i>
T	000
#	001
%	010
K	011
@	100
=	101
--- (Vacía)	110

*** (Especial)	111
----------------	-----

Teniendo en cuenta que para un tablero con filas (F) y columnas (C) se tienen $F \times C$ posiciones, cada ocupando tres bits, el tamaño en memoria requerido sería de $(3 \times F \times C)$; al igual, los bytes mínimos serían el resultado de $((3 \times F \times C)/8)$.

Para cada una de las posiciones se calcula primero su posición lógica, teniendo en cuenta que los índices comienzan en cero; así:

$$\text{Índice} = F \times C + C$$

$$\text{BitInicial} = 3 \times \text{Índice}$$

Luego, determinamos el byte asociado y el desplazamiento dentro de este, teniendo presente que, si el ‘Offset’ es mayor a cinco, el almacenamiento de la ficha se divide entre dos bytes:

$$\text{Byte} = \text{BitInicial} / 8$$

$$\text{Offset} = \text{BitInicial} \% 8$$

Dado al requerimiento de almacenamiento a nivel e bits, el tablero se almacenará mediante un ‘unsigned char*’; además, se evaluará la ocupación de memoria a medida que avanza el juego, de modo que, si esta llega a estar por debajo del 65% reservado inicialmente, se deberá redefinir su tamaño acorde a las nuevas medidas requeridas.

El programa funcionará siguiendo un algoritmo que permite leer unas dimensiones iniciales, hacer la respectiva gestión de memoria y presentar el tablero con sus fichas; proporcionar durante toda la duración del juego el menú con las acciones posibles; detectar combinaciones luego de cada acción, eliminarlas, rellenar los espacios en forma de cascada y

evaluar nuevas combinaciones, en caso de no haber se actualizan las estadísticas; finalmente, se repite el proceso desde el menú, hasta que el usuario decida finalizar el juego.

Diseño de la Solución

Se emplearán funciones para los segmentos de acciones que requiere el programa, estas serían:

Leer Ficha. Permite calcular el bit inicial, el byte y el offset, recibiendo el puntero al tablero y el índice; dependiendo de la posición, determina el desplazamiento que se debe hacer, la aplicación de máscaras y la combinación de bits mediante OR.

Almacenar Ficha. Permite limitar el código a tres bits, luego genera una mascara desplazada al offset hallado, los bits antiguos se eliminan mediante AND y los nuevos se incorporan mediante OR, en caso de que la ficha ocupe mas de un byte, se hace la división correspondiente.

Detectar Combinaciones. Para detectarlas combinaciones en cada fila y columna, se marcan las secuencias de tres o más fichas iguales mediante un arreglo dinámico.

Desplazamiento Vertical. Permite recorrer las columnas desde la parte de abajo desplazando hacia abajo las fichas y dejando marcación de los que queden vacíos.

Generar Fichas. Permite recorrer las posiciones marcadas como ‘vacías’ y generar códigos entre el grupo de símbolos asignados.

Efecto Cascada. Emplea la combinación del desplazamiento vertical, el relleno, la detección y la eliminación, siendo un ciclo que solo finaliza cuando no se encuentren más combinaciones

Modificación de Filas y columnas. Para el caso de requerir eliminar una fila o columna, se da una nueva distribución lógica, teniendo en cuenta que solo se modifica el tamaño de memoria si la ocupación está por debajo del 65% reservado, en caso contrario, solo hay una reorganización. Para agregar una fila o columna, se hace una nueva reserva con las dimensiones asociadas, además de recorrer el segmento anterior para para copiar las fichas que se conservan.

Se tiene en cuenta que los bits no utilizados del último byte quedan agrupados a la izquierda, mientras que los válidos comienzan desde el bit menos significativo y se debe hacer uso eficiente de la memoria dinámica, liberándola cuando sea necesario.

Dado a que por organización y eficiencia el programa debe tener estructura multi-archivo, se plantea implementar un archivo .h y .cpp para el tablero, donde se administre todo lo relacionado a la visualización de las fichas del tablero; otros para el juego, que permiten administrar las acciones que lo definen, como la detección de combinaciones; además, un main.cpp que gestiona la interacción con el usuario y la visualización de datos, integrando los archivos anteriores.